

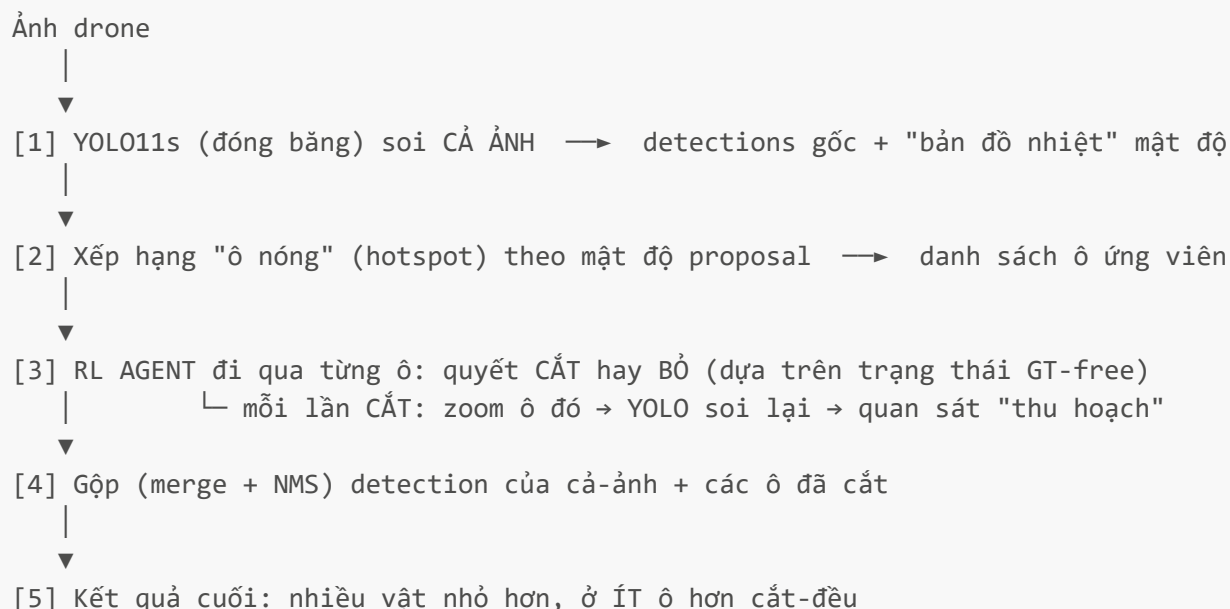
METHOD — Phương pháp RL-SAHI (chi tiết kỹ thuật)

Tài liệu này mô tả **chính xác** cách method hoạt động, lấy từ **code thật** trong `src/rl_sahi/`. Dùng cho phần "Phương pháp" của báo cáo và để team hiểu hệ thống. Method chính = **rl_yield** (Yield-aware Hotspot Agent). File code: `src/rl_sahi/rl/yield_env.py`, `yield_trainer.py`.

Mục lục

1. Tổng quan pipeline (5 bước)
2. Bước nền: density-guided (xếp hạng ô)
3. RL Agent — STATE (trạng thái, 15 chiều, GT-free)
4. RL Agent — ACTION (hành động)
5. RL Agent — REWARD (phần thưởng)
6. Huấn luyện (training)
7. Chạy thật (inference)
8. Đảm bảo GT-free (chống "ăn gian")
9. Bảng siêu tham số
10. Bản đồ code

1. Tổng quan pipeline (5 bước)



Điểm mấu chốt: YOLO **không bị sửa**. "Bộ não" RL chỉ quyết định **cắt ô nào** — đó là toàn bộ phần "học".

[↑ đầu](#)

2. Bước nền: density-guided (xếp hạng ô)

Trước khi agent quyết định, ta cần một **danh sách ô ứng viên** đã xếp hạng. Cách làm (`compute_static_features, state_maps.py`):

- 1. Chia ảnh thành lưới (vd 16×16).
- 2. Với mỗi ô, đếm **mật độ proposal** của YOLO (số box YOLO nghi có vật, kể cả độ tin cậy thấp).
- 3. Ô mật độ cao = nghi nhiều vật nhỏ → xếp hạng trên. Mỗi ô → 1 ROI vuông (cạnh = 35% cạnh ngắn của ảnh), tâm tại ô.
- 4. Loại ô trùng nhau (dedup theo khoảng cách tâm).

→ Agent sẽ duyệt các ô này theo thứ tự mật độ giảm dần. **density-guided cũng là baseline** mạnh để so (cắt top-K cố định).

↑ đầu

3. RL Agent — STATE (trạng thái, 15 chiều, GT-free)

Khi đứng trước ô thứ **i**, agent nhìn một vector **15 số** (`yield_state, yield_env.py:55`). Gồm 2 nhóm:

■ 8 đặc trưng TĨNH của ô (từ YOLO nhìn cả ảnh — không cần nhãn):

#	Đặc trưng	Ý nghĩa
1	<code>density</code>	Mật độ proposal trong ô (chuẩn hoá)
2	<code>objectness</code>	Điểm "có vật" của ô (từ feature map YOLO)
3	<code>cx/w</code>	Vị trí ngang của ô (0→1)
4	<code>cy/h</code>	Vị trí dọc của ô (0→1)
5	<code>dist_center</code>	Khoảng cách tới tâm ảnh (vật xa tâm thường nhỏ hơn)
6	<code>ncell/10</code>	Số proposal trong ô
7	<code>mean_conf</code>	Độ tin cậy trung bình của proposal trong ô
8	<code>max_conf</code>	Độ tin cậy cao nhất của proposal trong ô

■ 7 đặc trưng ĐỘNG (cập nhật khi agent đã cắt vài ô — đây là "trí nhớ"):

#	Đặc trưng	Ý nghĩa
9	<code>n_cropped/k</code>	Đã cắt bao nhiêu ô (so với tối đa)
10	<code>mean_yield</code>	Thu hoạch trung bình của các ô đã cắt (số vật mới quan sát)
11	<code>nearest_yield</code>	Thu hoạch của ô đã cắt gần ô hiện tại nhất
12	<code>max_yield</code>	Thu hoạch cao nhất từng thấy
13	<code>i/k</code>	Đang ở ô thứ mấy trong danh sách

#	Đặc trưng	Ý nghĩa
14	<code>remaining/k</code>	Còn bao nhiêu ô chưa duyệt
15	<code>skipped/k</code>	Đã bỏ qua bao nhiêu ô

🔑 **Vì sao GT-free:** nhóm động (9-15) dùng `raw_yield` = số box-mới YOLO sinh ra khi cắt (đếm được lúc chạy thật, **không cần nhãn**). Nhãn (GT) **chỉ** xuất hiện trong reward lúc train (mục 5). → Trạng thái lúc train **giống hệt** lúc thi → không có lỗi hỏng "học một đằng thi một nẻo". Có **test CI** chứng minh điều này (mục 8).

[↑ đầu](#)

4. RL Agent — ACTION (hành động)

Tại mỗi ô, agent chọn **1 trong 2** (`NUM_YIELD_ACTIONS = 2`):

Hành động	Mã	Làm gì
CROP	0	Cắt ô này → zoom → YOLO soi lại → nhận vật mới
SKIP	1	Bỏ qua ô này, sang ô tiếp

→ Agent duyệt hết danh sách ô (hoặc tới `k_max`). Số ô CẮT là **tự quyết theo từng ảnh** — đó chính là "phân bổ ngân sách thích nghi".

[↑ đầu](#)

5. RL Agent — REWARD (phần thưởng)

Công thức (`yield_env.py:151`), **chỉ dùng lúc train**:

```
Nếu CROP ô i:   reward = w_cov × real_yield[i] - crop_cost
Nếu SKIP:       reward = 0
```

- `real_yield[i]` = **số vật nhỏ THẬT (đúng nhãn) mới bắt được** nhờ cắt ô i (đây là chỗ duy nhất dùng GT, và chỉ lúc train).
- `w_cov` = thưởng cho mỗi vật bắt được. `crop_cost` = **phạt mỗi lần cắt** (để agent không cắt bừa).

💡 **Thực giác:** agent được thưởng khi cắt trúng ô nhiều vật, bị phạt cho mỗi nhát cắt → nó học **cắt ít mà trúng**. Đây là lý do nó đạt recall cao ở ít ô (xem REPORT §7).

[↑ đầu](#)

6. Huấn luyện (training)

Thuật toán: Deep Q-Network (DQN) — mạng MLP nhận state 15 chiều → ước lượng giá trị 2 hành động.

Tăng tốc bằng precompute cache: chạy YOLO trên mọi ô là rất chậm. Nên ta **tính trước** (`scripts/precompute_hotspot_yields.py`): với mỗi ô của mỗi ảnh train, lưu sẵn **raw_yield** (số box mới, GT-free) và **real_yield** (số vật thật, GT). → Lúc train agent **chỉ đọc cache**, không gọi YOLO → train cực nhanh (vài chục phút thay vì nhiều giờ).

Vòng train (mỗi episode = 1 ảnh):

1. Agent duyệt các ô, chọn CROP/SKIP bằng ϵ -greedy.
2. Nhận reward (từ **real_yield** trong cache).
3. Lưu (state, action, reward, next_state) vào replay buffer.
4. Cập nhật mạng Q (Double-DQN + n-step return).

Sanity check kiểu Karpathy: overfit 1 ảnh trước, theo dõi Q-value không phân kỳ, cố định seed.

[↑ đầu](#)

7. Chạy thật (inference)

Lúc chạy ảnh mới (`benchmark.py --yield-rl`), **không có cache, không có nhãn**:

1. YOLO soi cả ảnh → detections gốc + danh sách ô.
2. Agent duyệt từng ô, xem state (15 chiều **GT-free**) → quyết CROP/SKIP.
3. **Khi CROP:** zoom ô → gọi **YOLO thật** trên ô → đếm số box-mới → `set_observed_yield()` nạp con số này vào state (cập nhật nhóm động 9-12) cho ô tiếp theo.
4. Hết ô → gộp detections (cả-ảnh + các ô) bằng **class-aware NMS**.

→ Vì state lúc này **giống hệt** lúc train (đều dùng **raw_yield** quan sát được), agent hành xử nhất quán.

[↑ đầu](#)

8. Đảm bảo GT-free (chống "ăn gian" — điểm hội đồng chất vấn nặng nhất)

Nguyên tắc: trạng thái agent **tuyệt đối không chứa thông tin nhãn**. Nhãn chỉ vào **reward** lúc train.

- Nhóm động của state dùng **raw_yield** (đếm box, GT-free), **không** dùng **real_yield** (cần nhãn).
- Có **bài test tự động (CI)**: dựng 2 môi trường — một có nhãn, một không — và kiểm tra **state ra giống hệt nhau từng bit** (`tests/test_yield_env_state_gtfree.py`). Nếu lỗi rò nhãn vào state → test fail ngay.

→ Đây là cách trả lời câu "làm sao biết agent không gian lận khi vào ảnh mới?".

[↑ đầu](#)

9. Bảng siêu tham số (từ `configs/`)

Nhóm	Tham số	Giá trị
Detector	YOLO11s COCO, đóng băng · ảnh vào	640 px

Nhóm	Tham số	Giá trị
Detector	ngưỡng output · map lớp	0.25 · 6 lớp COCO→VisDrone
Ô cắt	cạnh ROI · số ô tối đa <code>k_max</code>	35% cạnh ngắn · 16
Reward	<code>w_cov</code> (thưởng/vật) · <code>crop_cost</code> (phạt/ô)	(config)
DQN	thuật toán	Double-DQN + n-step(3) + (PER tùy chọn)
DQN	optimizer · γ · batch · hidden	AdamW · 0.95 · 128 · 512
DQN	ϵ -decay · replay tối thiểu · reward clip	15000 bước · 512 · 10
Đo	tập · chỉ số	VAL 548 / TEST 1610 · mAP@0.5, recall, FP/ảnh, ô/ảnh

Mọi tham số nạp từ `configs/*.yaml` qua dataclass (`EnvConfig`, `StateConfig`, `TrainConfig`) — không hardcode.

[↑ đầu](#)

10. Bản đồ code (file → chức năng)

File	Chức năng
<code>src/rl_sahi/rl/yield_env.py</code>	Môi trường RL chính: state 15-chiều, action CROP/SKIP, reward
<code>src/rl_sahi/rl/yield_trainer.py</code>	Vòng train DQN (đọc precompute cache)
<code>scripts/precompute_hotspot_yields.py</code>	Tính trước <code>raw_yield/real_yield</code> mỗi ô (tăng tốc train)
<code>src/rl_sahi/rl/state_maps.py</code>	Xếp hạng ô theo mật độ + đặc trưng tĩnh
<code>src/rl_sahi/eval/benchmark.py</code>	Đo mAP/recall/FP, chạy <code>--yield-rl</code>
<code>tests/test_yield_env_state_gtfree.py</code>	Test CI: chứng minh state GT-free
<code>configs/*.yaml</code>	Mọi siêu tham số

Tóm 1 câu: YOLO đóng băng soi cả ảnh → xếp hạng ô nóng theo mật độ → **DQN agent quyết CẮT/BỎ từng ô dựa trên trạng thái GT-free (15 chiều) + trí nhớ thu hoạch**, được thưởng theo vật-thật-bắt-được trừ chi-phí-cắt → học **cắt ít ô mà trúng nhiều vật**. Chi tiết số liệu: xem [REPORT.md](#).